

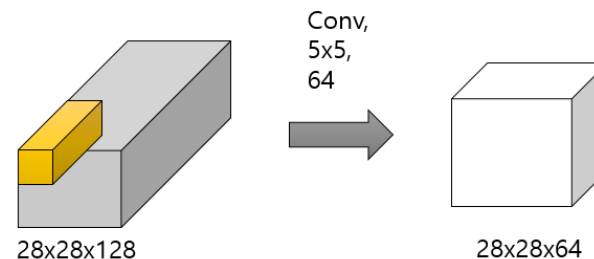
# 1x1 convolution

- 1x1 필터가 없는 경우

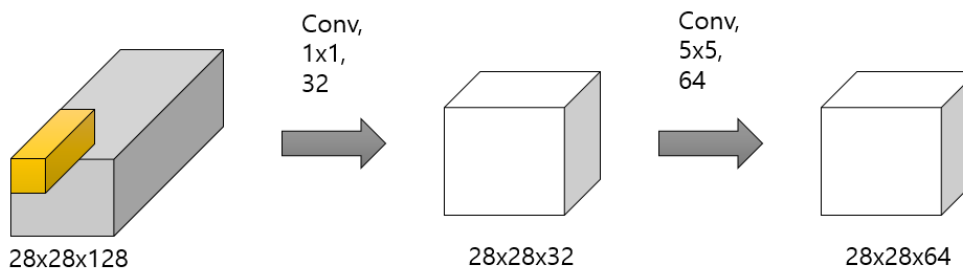
- ✓ **방식:** 처음부터 두꺼운 128채널 데이터에 5x5 필터를 바로 연산
- ✓ **연산량 계산:** 출력 크기 (28x28) x 출력 채널(64) x 필터 크기 (5x5) x 입력 채널(128)
- ✓ **결과:** 약 160M 연산 필요 → 필터 크기도 큰데 채널 수까지 많으니 컴퓨터가 계산해야 할 양이 폭발적으로 늘어남

- 1x1 필터로 '병목(Bottleneck)'을 만든 경우

- ✓ **Step A (차원 축소):** 가장 먼저 1 x 1 필터를 32개만 사용하여 가로세로 크기는 건드리지 않고, 채널만 128개에서 32개로 압축
  - 연산량:  $28 \times 28 \times 32 \times 128 \times 1 \times 1 = 4.8M$
- ✓ **Step B (본 연산):** 이제 32채널 데이터에 본래 목적이었던 5 x 5 필터 적용하여 최종 64 채널 생성(입력 채널이 128에서 32로 줄어든 상태라 연산이 가벼움)
  - 연산량:  $28 \times 28 \times 64 \times 5 \times 5 \times 32 = 40M$
- ✓ **결과:** 두 단계의 연산량을 합쳐도 Total = 44.8M



$$\#params = 28 \times 28 \times 64 \times 5 \times 5 \times 128 = 160M$$



$$\#params = 28 \times 28 \times 32 \times 128 \times 1 \times 1 = 4.8M$$

$$\#params = 28 \times 28 \times 64 \times 5 \times 5 \times 32 = 40M$$

$$\#total = 44.8M$$

- 1x1 필터 뒤 비선형 활성화함수의 경우

- ✓ 1x1 필터를 거치고 ReLU 활성화함수를 통과하면서 필요 없는 정보(음수 값 등)는 0으로 차단되고, 의미 있는 특징들만 새롭게 비선형적으로 조합
- ✓ 즉, 1x1 필터와 활성화함수의 결합은 데이터의 가로세로 특징을 뽑아내기 전에, 각 픽셀 위치에서 채널 간의 복잡한 관계를 먼저 한 번 딥러닝으로 학습시키는 역할

# Neural Style Transfer



- Neural Style Transfer (신경망 스타일 융합)

- ✓ 입력 사진의 객체 형태는 완벽히 보존하면서, 질감과 화풍만 지정된 명화(예: 빈센트 반 고흐)의 스타일로 다시 렌더링하는 기법
- ✓ 목적지 이미지  $G$  픽셀들을 타겟으로 삼고, 다음 오차 함수  $E(G)$ 가 바닥을 칠 때까지 경사 하강법으로 이미지 픽셀 값 자체를 직접 업데이트함
  - $C$  (Content image): 형태를 가져올 원본 사진 (예: 건물 사진)/ $S$  (Style image): 스타일을 가져올 명화 (예: 별이 빛나는 밤)/ $G$  (Generated image): 최적화 대상, 처음에는 랜덤 노이즈 또는  $C$ 의 복사본으로 시작
- ✓  $E(G) = E_{\text{content}}(G, C) + E_{\text{style}}(G, S)$  (콘텐츠 오차와 스타일 오차의 합)
  - $G$ 를 업데이트하여 이  $E(G)$ 를 최소화
  - $\eta$ 는 학습률이고, 미분은 이미지 픽셀에 대한 미분
  - 두 항의 비중을 조절하는 하이퍼파라미터  $\alpha, \beta$ 를 사용
  - $\alpha/\beta$  비율이 크면 원본 형태에 가깝고, 작으면 스타일이 더 강하게 반영

$$G \leftarrow G - \eta \cdot \frac{\partial E(G)}{\partial G}$$

$$E(G) = \alpha \cdot E_{\text{content}}(G, C) + \beta \cdot E_{\text{style}}(G, S)$$

- Content Error (형태 오차)

- ✓ CNN의 깊은 층은 고수준 특징(물체의 형태, 구조)을 인코딩. 따라서  $G$ 와  $C$ 를 같은 네트워크에 통과시켰을 때, 특정 깊은 층에서의 활성화 값이 비슷하면 형태가 비슷하다고 볼 수 있음
- ✓ 단순히 두 활성화 텐서 간의 L2 거리(제곱 오차)
- ✓  $a_{(ijk)}[l](G)$ : 생성 이미지  $G$ 를 네트워크에 넣었을 때  $l$ 층의 활성화
- ✓  $a_{(ijk)}[l](C)$ : 콘텐츠 이미지  $C$ 를 넣었을 때  $l$ 층의 활성화

$$E_{\text{content}}(G, C) = \frac{1}{2} \sum_{i,j,k} \left( a_{ijk}^{[l]}(G) - a_{ijk}^{[l]}(C) \right)^2$$

# Neural Style Transfer



- Style Error via Gram Matrix (스타일 오차)

- ✓ 스타일은 "어떤 패턴이 어디에 있는가"가 아니라, 어떤 패턴들이 함께 나타나는 경향이 있는가
- ✓  $F_{kk'}(G) = \sum_{i=1}^H \sum_{j=1}^W a_{ijk}(G) a_{ijk'}(G)$ : 공간 정보(위치  $i, j$ )를 모두 뭉개서 합해버린 후, 채널  $k$ 와 채널  $k'$  간 활성화 값이 동시에 얼마나 자주 증폭되었는지 나타내는 교차 상관 매트릭스 산출→채널 간 동시 발생 패턴을 포착하는 도구가 Gram Matrix
- ✓ 생성 맵  $G$ 의 스타일 텐서  $F(G)$ 와 명화  $S$ 의 스타일 텐서  $F(S)$  간의 통계적 차이를 줄이도록 제곱합 에러  $E_{style}$ 을 최적화함

$$F_{kk'}^{[l]}(G) = \sum_{i=1}^H \sum_{j=1}^W a_{ijk}^{[l]}(G) \cdot a_{ijk'}^{[l]}(G)$$

- ✓ 채널  $k$ 의 활성화 맵:  $H \times W$ 크기의 2D 맵 (예: "세로선 감지기")
- ✓ 채널  $k'$ 의 활성화 맵:  $H \times W$ 크기의 2D 맵 (예: "파란색 감지기")
- ✓ 같은 위치 ( $i, j$ )에서 두 값을 곱한 뒤 모든 위치에 대해 합산
- ✓ 값이 크다면 → 채널  $k$ 와  $k'$ 가 동시에 강하게 활성화되는 위치가 많다 → 두 특징이 함께 나타나는 경향이 강하다

- Style Loss 수식

- ✓ 생성 이미지와 스타일 이미지의 Gram Matrix가 같아지도록 최적화

$$E_{style} \propto \sum \{F(G) - F(S)\}^2$$

# Neural Style Transfer



- 전체 최적화 과정

$$G^* = \arg \min_G \left[ \alpha \sum_{i,j,k} \left( a_{ijk}^{[l]}(G) - a_{ijk}^{[l]}(C) \right)^2 + \beta \sum_l \lambda_l \sum_{k,k'} \left( F_{kk'}^{[l]}(G) - F_{kk'}^{[l]}(S) \right)^2 \right]$$

- ✓ G를 랜덤 노이즈로 초기화
- ✓ G, C, S를 pre-trained VGG에 통과시켜 각 층의 활성화를 구함
- ✓ Content Loss + Style Loss를 계산
- ✓ G의 픽셀에 대해 역전파하여 gradient 계산
- ✓ G의 픽셀 값을 업데이트
- ✓ 2~5를 수백~수천 회 반복
- ❖ 결과적으로 G는 C의 형태를 유지하면서 S의 질감/색상 패턴을 갖는 이미지로 수렴



# Deep Learning Foundations and Concepts 9주차

## 12. Transformers

김호중 / 2026.04.02



Computational Data Science LAB

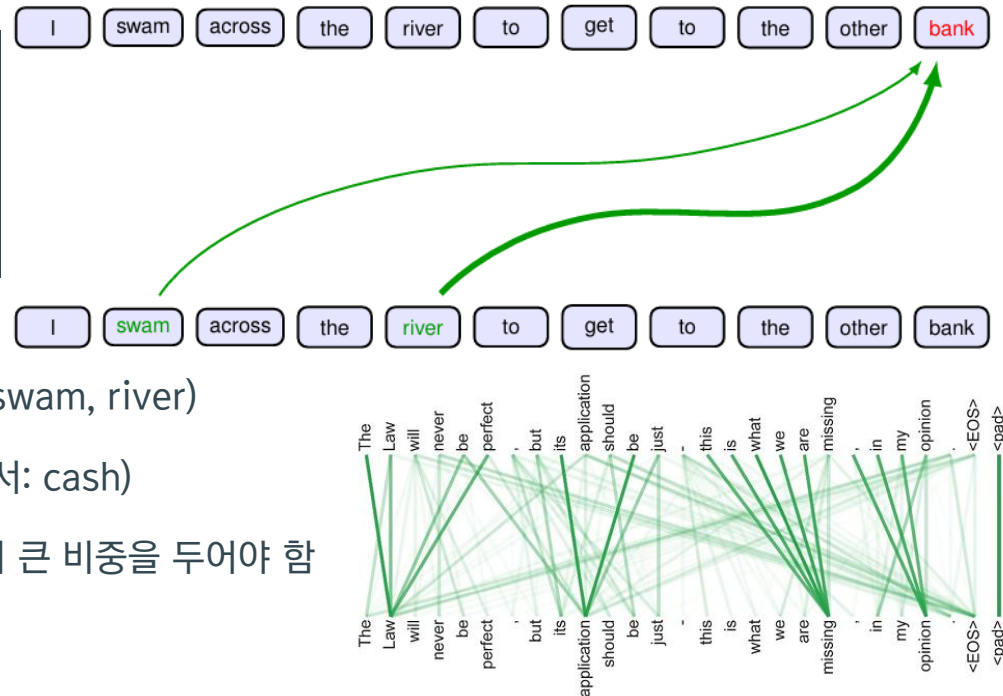
# CONTENTS

1. Why Attention?
2. Transformer Processing & Attention Coefficients
3. Self-Attention & Network Parameters
4. Scaled Multi-Head Attention
5. Transformer Layers & Computational Complexity
6. Positional Encoding
7. Natural Language: Word Embedding & Tokenization
8. Evolution of Language Models (BoW to RNN)
9. The Bottleneck Problem & Conclusion

# 01 | Why Attention?

## 문맥 의존적 표현과 Attention

- 동일 단어의 의미 변화 — 문맥(context)이 결정
  - ✓ "I swam across the river to get to the other bank." → bank = 강둑 (단서: swam, river)
  - ✓ "I walked across the road to get cash from the bank." → bank = 은행 (단서: cash)
  - ✓ bank라는 단어 자체만으로는 의미 판별 불가 → 주변 단어 중 관련성이 큰 단어에 더 큰 비중을 두어야 함
- 일반 신경망 vs Attention — 가중치의 본질적 차이
  - ✓ 일반 신경망: 학습 완료 후 가중치  $W$ 가 고정(*fixed*) → 어떤 입력이 와도 동일한 연결 강도
  - ✓ *Attention*: 가중치가 입력 데이터에 따라 동적(*dynamic*)으로 변화 → 입력 자체가 '어디를 볼지' 결정
- 정적 임베딩(Static Embedding) vs 문맥적 표현(Contextual Representation)
  - ✓ 기존 word embedding: 같은 단어 → 항상 동일 벡터 (문맥 무시). 예: bank → 항상 같은 벡터
  - ✓ Transformer: 주변 단어에 의존해 벡터 표현 자체가 변화 → 문맥적 표현 (Contextual Representation)
  - ✓ 첫 번째 문장: bank → water 방향으로 이동 / 두 번째 문장: bank → money 방향으로 이동
  - ✓ BERT, GPT 등 현대 모델이 강력한 이유 = 바로 이 문맥적 표현 능력에 기반

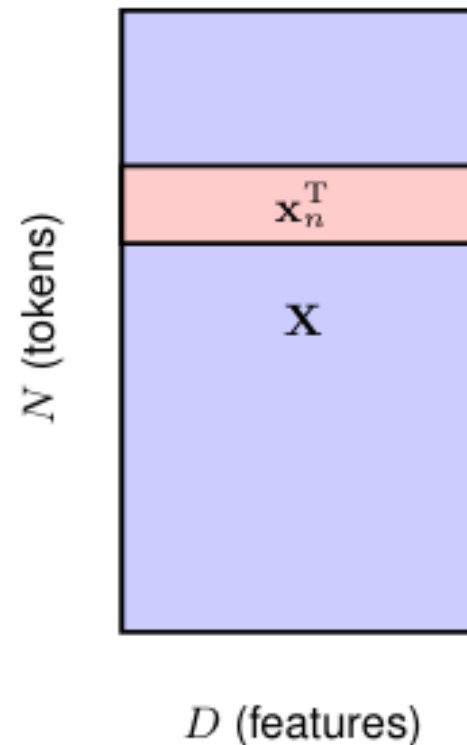




# 01 | Why Attention?

## 범용 메커니즘 & Transformer 입력 구조

- Attention은 언어 전용이 아니다 — 단백질 구조 예측 예시
  - ✓ 단백질: 아미노산의 1차원 서열이지만 실제로는 3차원 구조로 접힘
  - ✓ 서열상 멀리 떨어진 아미노산도 3D 공간에서는 인접 → 장거리 상호작용 발생
  - ✓ *Attention* = 멀리 떨어진 요소 간 중요한 관계를 유연하게 연결하는 범용 메커니즘
- Transformer의 입력: 토큰 벡터의 행렬 표현
  - ✓ 입력: N개의 벡터  $\{x_1, x_2, \dots, x_n\}$ , 각 벡터의 차원 = D
  - ✓ 토큰 벡터를 행으로 쌓으면 → 데이터 행렬  $X \in \mathbb{R}^{N \times D}$  (행 = token, 열 = feature)
  - ✓ 변수 정의
    - N = 시퀀스 내 토큰 개수 (문장 길이) | D = 각 토큰 벡터의 차원 (feature 수)
    - $x_n$  = n번째 토큰 벡터 (D차원) |  $X = [x_1^T; x_2^T; \dots; x_n^T] \in \mathbb{R}^{N \times D}$  (데이터 행렬)
  - ✓ 언어: 단어/subword → token, 비전: 이미지 패치 → token, 단백질: 아미노산 → token
  - ✓ 핵심 장점: 데이터 종류에 상관없이 토큰 벡터 집합으로 변환하면 동일한 구조 적용 가능





# 02 | Transformer Processing

## TransformerLayer 연산 & Attention Coefficients

- TransformerLayer: 핵심 연산 단위

$$\tilde{X} = \text{TransformerLayer}[X]$$

- ✓  $\tilde{X}$ : 출력 행렬 ( $\mathbb{R}^{N \times D}$ , 입력과 동일 크기) — 각 토큰이 문맥 정보를 반영한 새로운 표현
- ✓ 1단계 Attention: 토큰 간 정보 혼합 ("누가 누구를 참고?") / 2단계 MLP: 토큰 내부 feature 변환
- ✓ 여러 층 쌓기:  $X \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_L$  (깊은 표현 학습)

- Attention Coefficients: 출력 = 입력의 가중합

$$y_n = \sum_{m=1}^N a_{nm} x_m$$

- ✓  $y_n = n$ 번째 출력 토큰 벡터 (D차원) |  $x_m = m$ 번째 입력 토큰 벡터 (D차원)
- ✓  $a_{nm} = \text{attention weight}$  —  $y_n$ 을 만들 때  $x_m$ 에 부여하는 참조 비중 (클수록 많이 참고)

# 02 | Gradient Descent Optimization

## Attention Weight의 제약 조건

- Attention Weight의 두 가지 필수 제약

$$\begin{aligned} \textcircled{1} \quad & a_{nm} > 0 \\ \textcircled{2} \quad & \sum_{m=1}^N a_{nm} = 1 \end{aligned}$$

- ✓ 비음수: 어떤 토큰을 '반대로 참고'하는 해석 방지  $\rightarrow a_{nm}$ 은 영향력의 크기
- ✓ 합 = 1: 총 주의 예산(attention budget)이 고정  $\rightarrow$  하나를 많이 보면 다른 것은 덜 봄
- ✓ Partition of unity:  $a_{nm}$ 들은 입력 토큰들에 대한 '분배 계수'로 작동
- ✓  $a_{nn} = 1$ , 나머지 0  $\rightarrow y_n = x_n$  (자기 자신만 참고) / 여러 토큰에 분산  $\rightarrow$  blended representation

# 03 | Self-Attention & Network Parameters

## Dot-Product Self-Attention

- Query-Key-Value
  - ✓ Query (질의): 내가 찾고 싶은 조건 | Key (키): 검색 대상의 요약 정보 | Value (값): 가져올 실제 내용
  - ✓ 넷플릭스 비유: 사용자 취향 = Query, 영화 설명 = Key, 영화 파일 = Value
  - ✓ Hard attention: 하나만 선택 / Soft attention: 여러 후보를 부드럽게 혼합 → Transformer는 Soft (미분 가능)
- 가장 단순한 Self-Attention: 입력  $x_n$  자체를 Q, K, V 모두로 사용
  - ✓ 유사도 측정: dot product  $x_n^T x_m \rightarrow$  값이 클수록 두 토큰 간 관련성 높음
- $a_{nm} = \exp(x_n^T x_m) / \sum_{m'=1}^N \exp(x_n^T x_{m'})$  — *Softmax* 정규화
- $Y = \text{Softmax}(XX^T)X$  — 행렬식 표현
  - ✓  $XX^T$ : 토큰 간 유사도 행렬 |  $\text{Softmax}(XX^T)$ : 참조 비율 |  $\times X$ : 그 비율대로 정보 혼합
  - ✓ Q, K, V 모두 같은 시퀀스에서 생성 → Self-Attention, dot product 기반 → Dot-Product Self-Attention

# 03 | Self-Attention & Network Parameters

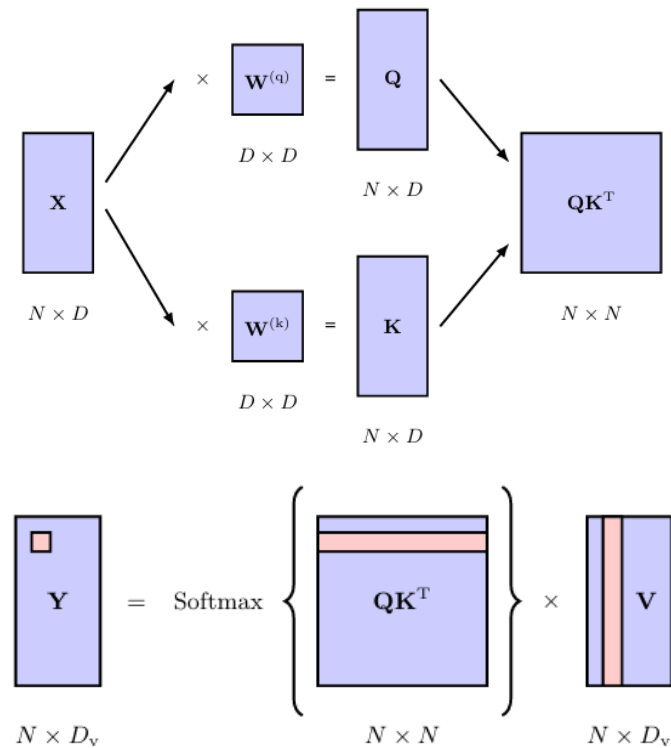
## Q, K, V 분리와 비대칭 관계 표현

- 단순 self-attention의 한계 → 학습 파라미터 없음 + 대칭 구조
  - ✓  $XU U^T X^T$  구조 → 본질적으로 대칭: A→B와 B→A 중요도가 동일하게 취급됨
  - ✓ 실제 의미 관계는 비대칭: "끝은 도구다" ≫ "도구는 끝이다" → 비대칭 표현 필요
- 세 개의 독립 선형변환 도입

$$Q = X W^{(q)} \quad K = X W^{(k)} \quad V = X W^{(v)}$$

$$Y = \text{Softmax}(QK^T) V$$

- ✓  $W^{(q)} \in \mathbb{R}^{D \times D_k}$  (Query 투영) |  $W^{(k)} \in \mathbb{R}^{D \times D_k}$  (Key 투영) |  $W^{(v)} \in \mathbb{R}^{D \times D_v}$  (Value 투영)
- ✓  $Q, K \in \mathbb{R}^{N \times D_k}$  (dot product를 위해 동일 차원) |  $V \in \mathbb{R}^{N \times D_v}$  |  $QK^T \in \mathbb{R}^{N \times N}$  (궁합 점수 행렬)



# 04 | Scaled Multi-Head Attention

## Scaled Dot-Product Self-Attention

- Softmax 포화 문제 & Scaling의 수학적 근거

- ✓  $QK^T$  원소의 절대값이 크면  $\rightarrow$  softmax 출력이 거의 one-hot  $\rightarrow$  gradient  $\approx 0$  (포화)
- ✓  $Q, K$  원소가 평균 0, 분산 1인 독립 확률변수일 때  $\rightarrow q_n^T k_m = \sum^{D_k} q_i k_i$  의 분산 =  $D_k$
- ✓ 따라서 표준편차  $\sqrt{D_k}$ 로 나누면  $\rightarrow$  softmax 입력의 분산이 1로 복원  $\rightarrow$  안정적 학습
- ✓ 현실의 데이터는 피쳐 간 스케일이나 상관관계 차이로 인해, 여러 표면이 거의 항상 조건수가 매우 높은 좁고 긴 계곡 형태를 띠

$$Y = \text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{D_k}) V$$

- ✓  $D_k$  = Query/Key 벡터의 차원 |  $\sqrt{D_k}$  = scaling factor (표준편차 보정)
- ✓  $QK^T / \sqrt{D_k} \in \mathbb{R}^{N \times N}$  = scaled attention score matrix  $\rightarrow$  Softmax 적용 후  $V$ 와 곱하여 최종 출력  $Y \in \mathbb{R}^{N \times D_v}$

### Algorithm 12.1: Scaled dot-product self-attention

**Input:** Set of tokens  $X \in \mathbb{R}^{N \times D} : \{x_1, \dots, x_N\}$

Weight matrices  $\{W^{(q)}, W^{(k)}\} \in \mathbb{R}^{D \times D_k}$  and  $W^{(v)} \in \mathbb{R}^{D \times D_v}$

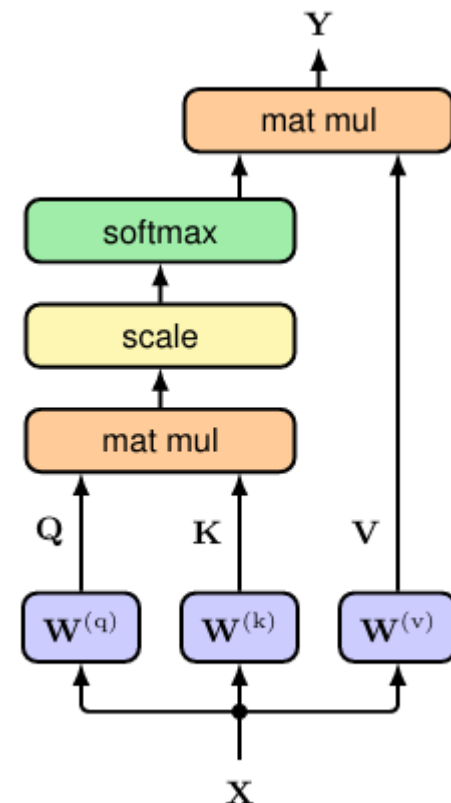
**Output:**  $\text{Attention}(Q, K, V) \in \mathbb{R}^{N \times D_v} : \{y_1, \dots, y_N\}$

$Q = XW^{(q)}$  // compute queries  $Q \in \mathbb{R}^{N \times D_k}$

$K = XW^{(k)}$  // compute keys  $K \in \mathbb{R}^{N \times D_k}$

$V = XW^{(v)}$  // compute values  $V \in \mathbb{R}^{N \times D_v}$

**return**  $\text{Attention}(Q, K, V) = \text{Softmax}\left[\frac{QK^T}{\sqrt{D_k}}\right] V$



# 04 | Scaled Multi-Head Attention

## Multi-Head Attention

- WHY Multi Head?

- ✓ 언어에는 문법, 시제, 장거리 참조, 의미 유사성 등 다양한 관계가 동시에 존재
- ✓ 한 Head만 쓰면 이런 관계들이 평균되어 소실 → CNN 다중 필터와 유사한 아이디어

$H_h = \text{Attention}(Q_h, K_h, V_h)$  — 각 head  $h$ 의 독립 attention

$$Q_h = XW_h^{(q)} \quad K_h = XW_h^{(k)} \quad V_h = XW_h^{(v)}$$

$Y(X) = \text{Concat}[H_1, H_2, \dots, H_h] W^{(o)}$  — 결과 이어붙이고 출력 투영

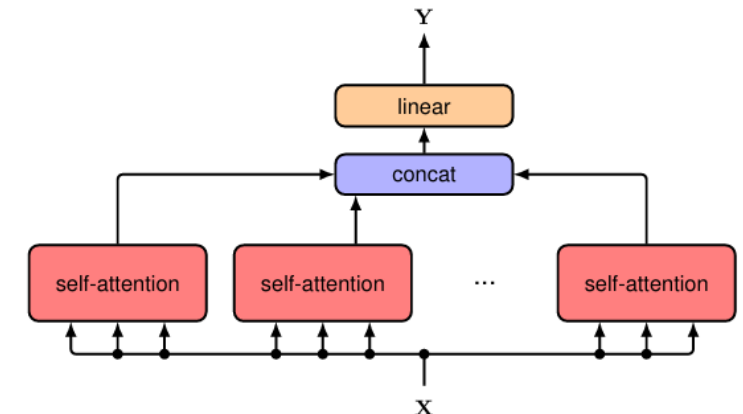
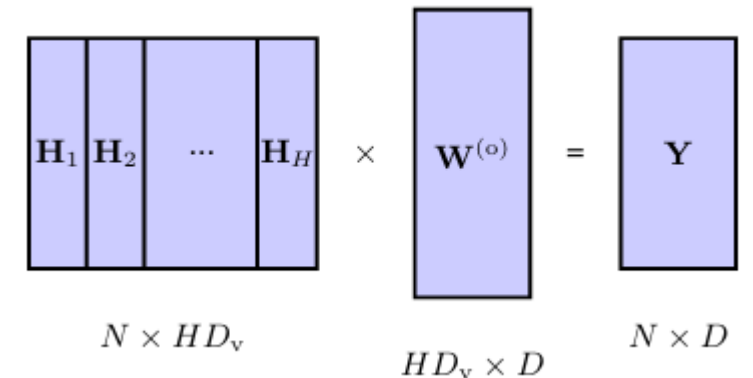
- ✓ 변수 정의:  $H$  = head 개수 |  $H_h \in \mathbb{R}^{N \times D_v} = h$ 번째 head의 attention 출력
- ✓  $W_h^{(q)}, W_h^{(k)}, W_h^{(v)} = \text{head별 독립 투영 행렬}$  |  $W^{(o)} \in \mathbb{R}^{HD_v \times D} = \text{출력 투영 (차원 복원)}$
- ✓  $\text{Concat}[H_1, \dots, H_h] \in \mathbb{R}^{N \times (HD_v)} \rightarrow W^{(o)}$  곱하여 원래 차원  $D$ 로 복원

Algorithm 12.2: Multi-head attention

**Input:** Set of tokens  $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$   
 Query weight matrices  $\{\mathbf{W}_1^{(q)}, \dots, \mathbf{W}_H^{(q)}\} \in \mathbb{R}^{D \times D}$   
 Key weight matrices  $\{\mathbf{W}_1^{(k)}, \dots, \mathbf{W}_H^{(k)}\} \in \mathbb{R}^{D \times D}$   
 Value weight matrices  $\{\mathbf{W}_1^{(v)}, \dots, \mathbf{W}_H^{(v)}\} \in \mathbb{R}^{D \times D_v}$   
 Output weight matrix  $\mathbf{W}^{(o)} \in \mathbb{R}^{HD_v \times D}$

**Output:**  $\mathbf{Y} \in \mathbb{R}^{N \times D} : \{\mathbf{y}_1, \dots, \mathbf{y}_N\}$

```
// compute self-attention for each head (Algorithm 12.1)
for  $h = 1, \dots, H$  do
     $Q_h = XW_h^{(q)}$ ,  $K_h = XW_h^{(k)}$ ,  $V_h = XW_h^{(v)}$ 
     $H_h = \text{Attention}(Q_h, K_h, V_h)$  //  $H_h \in \mathbb{R}^{N \times D_v}$ 
end for
 $H = \text{Concat}[H_1, \dots, H_N]$  // concatenate heads
return  $Y(X) = HW^{(o)}$ 
```



# 05 | Transformer Layers & Complexity

## Residual Connection, LayerNorm, MLP

- Attention 블록 + Residual + LayerNorm

Post-norm:  $Z = \text{LayerNorm}[Y(X) + X]$

Pre-norm:  $Z = Y(X') + X, X' = \text{LayerNorm}[X]$

MLP 블록:  $X' = \text{LayerNorm}[MLP[Z] + Z]$

Pre-norm:  $X' = MLP(Z') + Z, Z' = \text{LayerNorm}[Z]$

### Algorithm 12.3: Transformer layer

**Input:** Set of tokens  $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

Multi-head self-attention layer parameters

Feed-forward network parameters

**Output:**  $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D} : \{\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N\}$

$Z = \text{LayerNorm}[Y(\mathbf{X}) + \mathbf{X}]$  //  $Y(\mathbf{X})$  from Algorithm 12.2

$\tilde{\mathbf{X}} = \text{LayerNorm}[MLP[Z] + Z]$  // shared neural network

return  $\tilde{\mathbf{X}}$

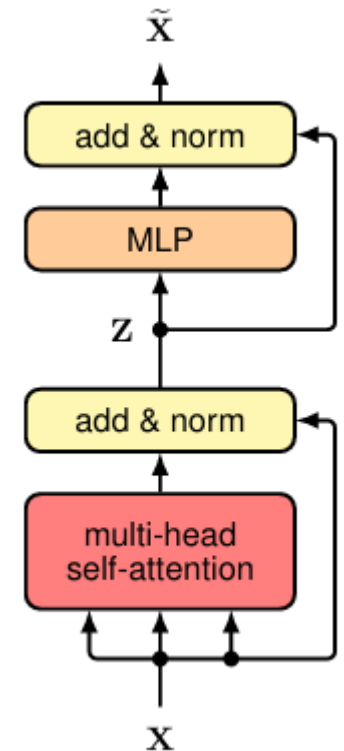
✓  $Y(X)$  = Multi-Head Attention 출력 | + X = residual connection (skip) | MLP = 2층 FC + ReLU

✓ Pre-norm이 최적화에 더 유리한 경우가 있음 (GPT 계열에서 주로 사용)

- WHY MLP?

✓ Attention 출력 = value 벡터들의 선형결합(가중합) → 입력이 span하는 부분공간에 갇힘

✓ MLP가 각 토큰별로 비선형 변환 적용 → 표현력 확장. 동일 MLP를 모든 토큰에 공유 (길이 무관)





# 06 | Positional Encoding

## Permutation Equivariance & Sinusoidal PE

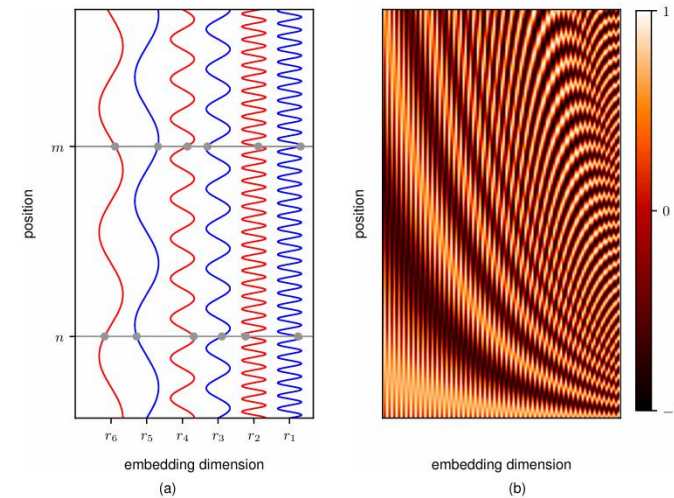
- 왜 위치 정보가 필요한가? — *Permutation Equivariance*
  - ✓ 현재 구조는 토큰 순서를 바꾸면 출력도 그 순서만 바뀜 → 순서 자체를 인식하지 못함
  - ✓ "The food was bad, not good at all." vs "The food was good, not bad at all." → 의미 정반대
  - ✓ Solution: 구조를 바꾸지 않고 입력 토큰에 위치벡터를 더함 (add)

$$\tilde{x}_n = x_n + r_n \quad x_n: \text{토큰 임베딩}, r_n: \text{위치 인코딩 벡터 (D차원)}$$

- Sinusoidal Positional Encoding

$$r_{ni} = \sin(n / L^{(i/D)}) \quad (i \text{가 짝수}) \quad r_{ni} = \cos(n / L^{((i-1)/D)}) \quad (i \text{가 홀수})$$

- ✓  $n$  = 시퀀스 내 위치 (0,1,2,...) |  $i$  = 임베딩 차원 인덱스 (0~D-1) |  $D$  = 임베딩 차원 |  $L$  = 주기 조절 상수
- ✓ 낮은  $i$  → 빠르게 진동 (하위 비트) / 높은  $i$  → 천천히 진동 (상위 비트) — 이진수 표현과 유사



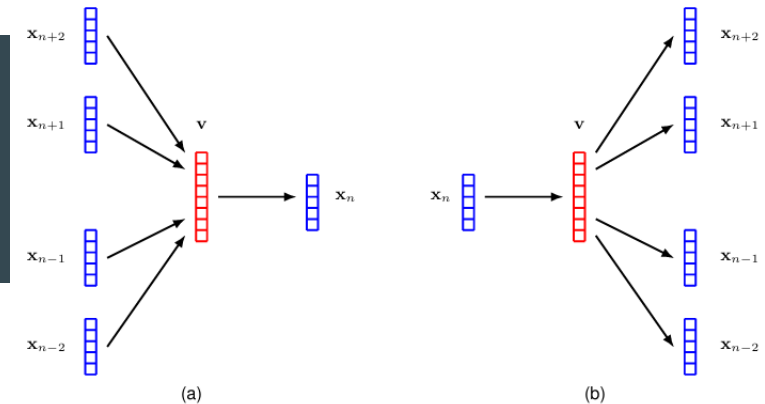
# 06 | Positional Encoding

## Add & PE

- It's Not Concat But Add
  - ✓ 고차원 공간에서 무작위 벡터들은 거의 직교  $\rightarrow$  *token identity*와 *position info*가 분리 처리됨
  - ✓ Residual connection 덕분에 position 정보가 깊은 층에서도 사라지지 않음
  - ✓ 선형변환 관점에서 concat과 add는 비슷한 성질 보유 + 차원 증가 없음
- Sinusoidal PE의 핵심 성질: 상대 위치(Relative Position) 표현
  - ✓ 임의의 고정 offset  $k$ 에 대해:  $PE(n+k) = f(PE(n), k)$  ( $PE(n)$ 의 선형결합, 계수는  $k$ 에만 의존)
  - ✓ 모델이 "2칸 뒤", "5칸 앞" 같은 상대위치를 학습하기 용이
  - ✓ sine만으로는 불가능  $\rightarrow$  cosine 쌍이 반드시 필요 (삼각함수 덧셈정리 활용)
- 좋은 PE의 조건 vs 부적절한 대안
  - ✓ 각 위치 고유(unique) / 값 bounded / 미학습 긴 시퀀스 일반화 / 상대위치 일관 표현
  - ✓ 정수 1,2,3...: unbounded  $\rightarrow$  임베딩 왜곡 / 0~1 비율: 길이 바뀌면 같은 위치도 다른 값
  - ✓ Learned PE: hand-crafted 불필요하나 학습 시 못 본 긴 위치에 일반화 약함

# 07 | Natural Language

## Word Embedding & word2vec



- 언어 → 숫자 변환: One-hot → Dense Embedding
  - ✓ One-hot:  $(0,0,1,0,...,0)$  — 차원 = dictionary 크기(수십만), 단어 간 유사성 반영 불가
  - ✓ Solution: 저차원 dense vector로 매핑하는 Word Embedding

$$v_n = E x_n \quad E \in \mathbb{R}^{D \times K}, x_n = \text{one-hot} \rightarrow E \text{의 해당 열 추출}$$

- ✓  $v_n = n$ 번째 단어의 임베딩 벡터 (D차원) |  $E$  = 임베딩 행렬 |  $K$  = dictionary 크기 |  $D$  = 임베딩 차원
- word2vec: Self-supervised Embedding
  - ✓ CBOW (Continuous Bag of Words): 주변 M개 단어 → 가운데 단어 예측 (빈칸 채우기)
  - ✓ Skip-gram: 가운데 단어 → 주변 단어 예측 (이 단어 주변에 뭐가 올까?)
  - ✓ Self-supervised: 라벨 없이 텍스트 자체에서 입력/정답 생성 → 대규모 코퍼스로 학습
- 벡터 산술: 관계 구조가 임베딩 공간에 인코딩됨

$$v(\text{Paris}) - v(\text{France}) + v(\text{Italy}) \approx v(\text{Rome}) \quad \text{"수도 : 나라" 관계 = 벡터 차이}$$

- ✓ 비슷한 문맥에서 등장하는 단어가 임베딩 공간에서 가까워짐 (통계적 분포 유사성에 기반)

# 07 | Natural Language

## Tokenization & Byte Pair Encoding

- 고정 단어 사전의 한계
  - ✓ OOV(미등록어) 처리 불가, 오타 · 코드 · 특수문자 약함
  - ✓ 문자 단위는 단어 구조 무너짐 + 시퀀스 과도하게 길어짐
  - ✓ Solution: 서브워드(Subword) 토큰 — 흔한 단어는 통째로, 드문 단어는 조각으로 분할
  - ✓ Ex) cook, cooks, cooked, cooking → 모두 'cook' 조각 공유
- Byte Pair Encoding (BPE) — 압축 알고리즘에서 유래
  - ✓ 1단계: 모든 문자를 개별 토큰으로 초기화 (alphabet level)
  - ✓ 2단계: 코퍼스 전체에서 가장 빈번한 인접 토큰쌍 탐색
  - ✓ 3단계: 그 쌍을 하나의 새 토큰으로 합침 (단어 경계 넘지 않음)
  - ✓ 4단계: 원하는 vocabulary 크기에 도달할 때까지 반복

Peter Piper picked a peck of pickled peppers  
Peter Piper picked a peck of pickled peppers  
Peter Piper picked a peck of pickled peppers  
Peter Piper picked a peck of pickled peppers  
Peter Piper picked a peck of pickled peppers  
Peter Piper picked a peck of pickled peppers

- 문자 → pe → ck → pi → ed → per 순서로 병합  
→ vocabulary 크기 ↔ 표현력 절충

# 08 | Evolution of Language Models

## Bag-of-Words & Autoregressive Models

- Bag-of-Words: 가장 단순한 확률 모형 — 순서 완전 무시

$$p(x_1, \dots, x_n) = \prod_{n=1}^N p(x_n)$$

- ✓ "dog bites man" = "man bites dog" 동일 취급 → 순서 정보 완전 손실
- ✓ Naive Bayes 텍스트 분류:  $p(C_k | x_1, \dots, x_n) \propto p(C_k) \prod_n p(x_n | C_k)$

- Autoregressive 분해: 순서를 조건부곱으로 표현 — GPT

$$p(x_1, \dots, x_n) = \prod_{n=1}^N p(x_n | x_1, \dots, x_{n-1}) \quad \leftarrow \text{GPT의 수학적 근본}$$

- ✓ 앞에 나온 모든 단어 조건 → 다음 단어 분포 예측. 완전히 일반적인 분해 (정보 손실 없음)
- ✓ 문제: 조건 길이 ↑ → 경우의 수 지수적 폭증 → 확률표 직접 저장 불가능

- N-gram: 바로 앞 L개 단어만 조건으로 사용하는 단순화

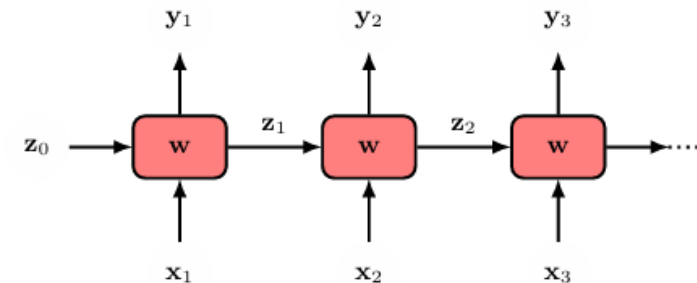
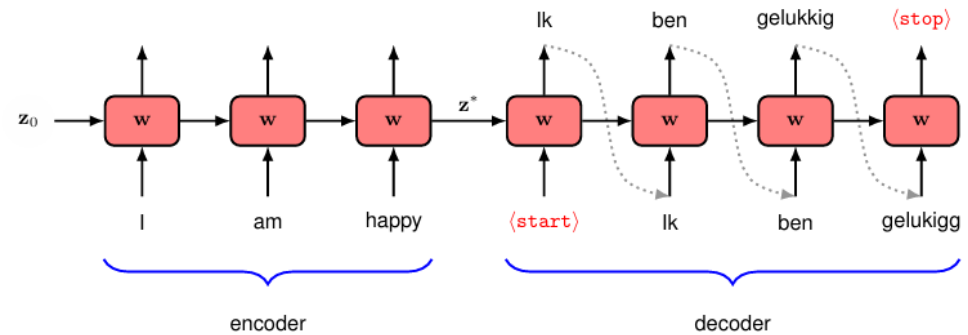
$$L = 2 \text{ (tri-gram)}: p(x_1, \dots, x_n) = p(x_1) p(x_2 | x_1) \prod_{n=3} p(x_n | x_{n-1}, x_{n-2})$$

- ✓  $L \uparrow \rightarrow$  표 크기 지수 폭발. 핵심: autoregressive 분해 자체는 중요 — 현대 LLM은 '표' 대신 '딥러닝'으로 조건부분포 모델링

# 08 | Evolution of Language Models

## Recurrent Neural Networks & Encoder-Decoder

- Feed-forward의 한계 → RNN의 필요성
  - ✓ FC network: 입출력 수 고정 → 가변 길이 문장 처리 불가
  - ✓ 시퀀스의 다른 위치에서 같은 패턴은 비슷한 의미 → 위치별 다른 파라미터는 비효율
- RNN
  - ✓ 각 시점: 현재 입력  $x_n$  + 이전 hidden state  $z_{n-1}$  → 출력  $y_n$  + 다음 hidden state  $z_n$
  - ✓ 동일한 셀(동일 파라미터  $w$ )을 시간축으로 반복 연결 → 길이 무관 적용 가능
  - ✓  $z_0 = 0$  벡터(초기 상태). CNN의 weight sharing과 유사한 equivariance 아이디어
- Encoder-Decoder 구조
  - ✓ Encoder: 입력 문장 전체를 끝까지 읽으며 hidden state에 의미 압축
  - ✓ Decoder:  $\langle \text{start} \rangle$  토큰 입력 후 hidden state 기반으로 출력 단어를 하나씩 생성 →  $\langle \text{stop} \rangle$
  - ✓ 생성된 출력 단어를 다시 다음 시점의 입력으로 넣음 → Autoregressive 생성 구조



# 09 | The Bottleneck Problem & Conclusion

## RNN의 근본적 한계 → Transformer의 등장

- Backpropagation Through Time (BPTT)
  - ✓ 시간축으로 펼친 네트워크를 따라 error signal 역전파
  - ✓ 시퀀스 길어질수록 gradient 반복 곱셈 → Vanishing/Exploding Gradient
- Bottleneck Problem (정보 병목)
  - ✓ 전체 입력 문장의 의미를 하나의 고정 길이 hidden vector  $z^*$ 에 압축해야 함
  - ✓ 긴 문장일수록 중요 정보 손실 불가피 → 정보가 병목 지점을 통과하며 소실
- LSTM, GRU: 부분적 개선이나 근본 해결 아님
  - ✓ Gate 메커니즘으로 gradient 경로 확보 → 장거리 기억 개선
  - ✓ 그러나: ① 여전히 장거리 의존성 한계 ② 셀 복잡 → 느림 ③ 본질적 순차 처리 → GPU 병렬화 불가
- Conclusion: RNN의 세 가지 근본적 한계
  - ✓ ① 순차 처리 (병렬화 불가) ② 장거리 관계 포착 한계 ③ 정보 병목 (고정 크기 벡터 압축)
  - ✓ → 이 세 문제를 동시에 해결하는 구조가 바로 Transformer (Attention만으로 구성, 완전 병렬 처리 가능)

# Q&A

---

감사합니다.